AMS 526: Numerical Linear Algebra for Computational and Data Sciences

Lecture 13: Stability of Householder Triangularization; Other Methods for Least Squares Problems; Linear Algebra Software

Prof. Xiangmin Jiao

Stony Brook University

Outline

1. **Stability of Householder Triangularization (NLA §16,19)**
2. Solution of Least Squares Problems
3. Rank-Deficient Least Squares Problems
4. Software for Linear Algebra

Solution of Least Squares Problems

- An efficient and robust approach is to use QR factorization $A = \hat{Q}\hat{R}$.
  - b can be projected onto range(A) by $P = \hat{Q}\hat{Q}^{\mathsf{T}}$, and therefore $\hat{Q}\hat{R}x = \hat{Q}\hat{Q}^{\mathsf{T}}b$.
  - Left-multiply by $\hat{Q}^{\mathsf{T}}$ and we get $\hat{R}x = \hat{Q}^{\mathsf{T}}b$ (note that $A^{+} = \hat{R}^{-1}\hat{Q}^{\mathsf{T}}$).

Least squares via QR Factorization

Compute reduced QR factorization $A = \hat{Q}\hat{R}$.
Compute vector $c = \hat{Q}^{\mathsf{T}}b$.
Solve upper-triangular system $\hat{R}x = c$ for x.

- Computation is dominated by the QR factorization ($2mn^2 - \frac{2}{3}n^3$ flops).
- What about stability?

Backward Stability of Householder Triangularization

- For a QR factorization $A = QR$ computed by Householder triangularization, the factors $\tilde{Q}$ and $\tilde{R}$ satisfy

$$\tilde{Q}\tilde{R} = A + \delta A, \quad \frac{\|\delta A\|}{\|A\|} = \mathcal{O}(\epsilon_{\text{machine}}),$$

  i.e., exact QR factorization of a slightly perturbed A.
- $\tilde{R}$ is the R computed by the algorithm using floating point arithmetic.
- However, $\tilde{Q}$ is the product of *exactly orthogonal* reflectors

$$\tilde{Q} = \tilde{Q}_1 \tilde{Q}_2 \dots \tilde{Q}_n,$$

  where $\tilde{Q}_k$ is given by the computed $\tilde{v}_k$, since Q is not formed explicitly.

Backward Stability of Solving $Ax = b$ with QR

Algorithm: Solving $Ax = b$ by QR Factorization

Compute $A = QR$ using Householder, represent Q by reflectors.
Compute vector $y = Q^\top b$ implicitly using reflectors.
Solve upper-triangular system $R_{1:n,1:n}x = y_{1:n}$ for x.

- All three steps are backward stable.
- Overall, we can show that

$$(A + \Delta A)\tilde{x} = b, \quad \frac{\|\Delta A\|}{\|A\|} = \mathcal{O}(\epsilon_{\text{machine}}).$$

Backward Stability of Solving $Ax = b$ with Householder Triangulariza

Proof: Step 2 gives

$$(\tilde{Q} + \delta Q)\tilde{y} = b, \quad \|\delta Q\| = \mathcal{O}(\epsilon_{\text{machine}}).$$

Step 3 gives

$$(\tilde{R} + \delta R)\tilde{x} = \tilde{y}, \quad \frac{\|\delta R\|}{\|\tilde{R}\|} = \mathcal{O}(\epsilon_{\text{machine}}).$$

Therefore,

$$b = (\tilde{Q} + \delta Q)(\tilde{R} + \delta R)\tilde{x} = \left[\tilde{Q}\tilde{R} + (\delta Q)\tilde{R} + \tilde{Q}(\delta R) + (\delta Q)(\delta R)\right]\tilde{x}.$$

Step 1 gives

$$b = \left[A + \underbrace{\delta A + (\delta Q)\tilde{R} + \tilde{Q}(\delta R) + (\delta Q)(\delta R)}_{\Delta A}\right]\tilde{x},$$

where $\tilde{Q}\tilde{R} = A + \delta A$.

Proof of Backward Stability Cont'd

We have $\tilde{Q}\tilde{R} = A + \delta A$ where $\frac{\|\delta A\|}{\|A\|} = \mathcal{O}(\epsilon_{\text{machine}})$, and therefore

$$\frac{\|\tilde{R}\|}{\|A\|} \leq \|\tilde{Q}^{\mathsf{T}}\| \frac{\|A + \delta A\|}{\|A\|} = \mathcal{O}(1).$$

We now show that each term in ΔA is small:

$$\frac{\|(\delta Q)\tilde{R}\|}{\|A\|} \leq \|\delta Q\| \frac{\|\tilde{R}\|}{\|A\|} = \mathcal{O}(\epsilon_{\text{machine}}),$$

$$\frac{\|\tilde{Q}(\delta R)\|}{\|A\|} \leq \|\tilde{Q}\| \frac{\|\delta R\|}{\|\tilde{R}\|} \frac{\|\tilde{R}\|}{\|A\|} = \mathcal{O}(\epsilon_{\text{machine}}),$$

$$\frac{\|(\delta Q)(\delta R)\|}{\|A\|} \leq \|\delta Q\| \frac{\|\delta R\|}{\|A\|} = \mathcal{O}(\epsilon_{\text{machine}}^2).$$

Overall,

$$\frac{\|\Delta A\|}{\|A\|} \leq \frac{\|\delta A\|}{\|A\|} + \frac{\|(\delta Q)\tilde{R}\|}{\|A\|} + \frac{\|\tilde{Q}(\delta R)\|}{\|A\|} + \frac{\|(\delta Q)(\delta R)\|}{\|A\|} = \mathcal{O}(\epsilon_{\text{machine}}).$$

Since the algorithm is backward stable, it is also accurate.

Backward Stability of Householder Triangularization

Theorem

Let the full-rank least squares problem be solved using Householder triangularization on a computer satisfying the two axioms of floating point numbers. The algorithm is backward stable in the sense that the computed solution $\tilde{x}$ has the property

$$\|(A + \Delta A)\tilde{x} - b\| = \min, \quad \frac{\|\Delta A\|}{\|A\|} = \mathcal{O}(\epsilon_{\text{machine}})$$

for some $\Delta A \in \mathbb{R}^{m \times n}$.

- Backward stability of the algorithm is true whether $\hat{Q}^{\mathsf{T}} b$ is computed via explicit formation of $\hat{Q}$ or computed implicitly.
- Backward stability also holds for Householder triangularization with arbitrary column pivoting $AP = \hat{Q}\hat{R}$.

Outline

1. Stability of Householder Triangularization (NLA §16,19)
2. **Solution of Least Squares Problems**
3. Rank-Deficient Least Squares Problems
4. Software for Linear Algebra

Algorithms for Solving Least Squares Problems

- There are many variants of algorithms for solving least squares problems:
  - Householder QR (with/without pivoting, explicit or implicit Q): **Backward stable**.
  - Classical Gram–Schmidt: **Unstable**.
  - Modified Gram–Schmidt with explicit Q: **Unstable**.
  - Modified Gram–Schmidt with augmented system of equations with implicit Q: **Backward stable**.
  - Normal equations (solve $A^\top Ax = A^\top b$): **Unstable**.
  - Singular value decomposition: **Stable and most accurate**.

Stability of Gram–Schmidt Orthogonalization

- Gram–Schmidt QR is unstable, due to loss of orthogonality.
- Gram–Schmidt can be stabilized using an augmented system of equations:
  1. Compute QR factorization of the augmented matrix: $[Q, R_1] = \text{mgs}([A, b])$.
  2. Extract R and $\hat{Q}^T b$ from R_1:

$$R = R_1(1:n, 1:n) \quad \text{and} \quad Qb = R_1(1:n, n+1).$$

  3. Back solve: $x = R \backslash Qb$.

Theorem

The solution of the full-rank least squares problem by Gram–Schmidt orthogonalization is backward stable in the sense that the computed solution $\tilde{x}$ has the property

$$\|(A + \Delta A)\tilde{x} - b\| = \min, \quad \frac{\|\Delta A\|}{\|A\|} = \mathcal{O}(\epsilon_{\text{machine}})$$

for some $\Delta A \in \mathbb{R}^{m \times n}$, provided that $\hat{Q}^T b$ is formed implicitly.

Other Methods

- The method of *normal equations* solves $x = (A^\top A)^{-1} A^\top b$, which is unstable due to the squaring of the condition number of A.

Theorem

The solution of the full-rank least squares problem via normal equations is unstable. Stability can be achieved, however, by restriction to a class of problems in which $\kappa(A)$ is uniformly bounded above.

- Another method is to use SVD.

Solution by SVD

- Using $A = \hat{U}\hat{\Sigma}V^{\mathsf{T}}$, b can be projected onto $\mathrm{range}(A)$ by $P = \hat{U}\hat{U}^{\mathsf{T}}$, and therefore $\hat{U}\hat{\Sigma}V^{\mathsf{T}}x = \hat{U}\hat{U}^{\mathsf{T}}b$.
- Left-multiply by $\hat{U}^{\mathsf{T}}$ and we get $\hat{\Sigma}V^{\mathsf{T}}x = \hat{U}^{\mathsf{T}}b$.

Least squares via SVD

Compute reduced SVD factorization $A = \hat{U}\hat{\Sigma}V^{\mathsf{T}}$.
Compute vector $c = \hat{U}^{\mathsf{T}}b$.
Solve diagonal system $\hat{\Sigma}w = c$ for w.
Set $x = V_{1:n,1:r}w$.

- Work is dominated by SVD, which is $\sim 2mn^2 + 11n^3$ flops, and is very expensive if $m \approx n$.
- Question: If A is rank deficient, how do we solve $Ax \approx b$?

Outline

1. Stability of Householder Triangularization (NLA §16,19)
2. Solution of Least Squares Problems
3. **Rank-Deficient Least Squares Problems**
4. Software for Linear Algebra

Rank-Deficient Least Squares Problems

- Least squares problems $Ax \approx b$ are the most challenging if A is (nearly) rank deficient.
- If A is rank deficient, there are an infinite number of x that minimize $\|b - Ax\|$. This is because if $y \in \mathrm{null}(A)$, for any x that minimizes $\|b - Ax\|$, $x + y$ also minimizes $\|b - Ax\|$.
- "Uniqueness" is recovered by requiring $x \perp \mathrm{null}(A)$, or equivalently, minimizing $\|x\|$ subject to $(b - Ax) \perp \mathrm{range}(A)$.
- In practice, however, we often have near rank deficiency instead of exact rank deficiency.
- If A is rank deficient, the (left or right) null space is the space spanned by (left or right) singular vectors corresponding to zero singular values.
- For nearly rank deficient least squares problems, define the "numerical null space" to be the singular vectors corresponding to the smallest singular values.

Solving Rank-Deficient Least Squares Problems by SVD

- If A is full rank, $A = \hat{U}\hat{\Sigma}V^{\mathsf{T}} = \sum_{j=1}^{\min\{m,n\}} \sigma_j u_j v_j^{\mathsf{T}}$, and $A^{+} = \sum_{j=1}^{\min\{m,n\}} \frac{1}{\sigma_j} v_j u_j^{\mathsf{T}}$.
- If A is rank deficient, $A^{+} = \sum_{j=1}^{r} \frac{1}{\sigma_j} v_j u_j^{\mathsf{T}}$, where r is the rank of A.
- If A is nearly rank deficient, $\tilde{A}^{+} = \sum_{j=1}^{r} \frac{1}{\sigma_j} v_j u_j^{\mathsf{T}}$, where r is the *numerical rank* of A, i.e., the largest j such that $\sigma_j \geq \epsilon\sigma_1$ for some small ϵ. This is called *truncated SVD*.
- $\tilde{A} = \sum_{j=1}^{r} \sigma_j u_j v_j^{\mathsf{T}}$ is a low-rank approximation to A.

Rank-deficient least squares via truncated SVD

Compute reduced SVD factorization $A = \hat{U}\hat{\Sigma}V^{\mathsf{T}}$ and estimate r.
Compute vector $c = \left(\hat{U}_{:,1:r}\right)^{\mathsf{T}} b$.
Solve diagonal system $\hat{\Sigma}_{1:r,1:r} w = c$ for w.
Set $x = V_{1:n,1:r} w$.

A Note on Pseudoinverse

- If $A \in \mathbb{R}^{m \times n}$ is rank deficient, the pseudoinverse is

$$A^{+} = \sum_{j=1}^{r} \frac{1}{\sigma_j} v_j u_j^{\top},$$

  where r is the rank of A.

- It is the unique minimum Frobenius norm solution to

$$\min_{X \in \mathbb{R}^{n \times m}} \|AX - I_m\|_F,$$

  where I_m is the $m \times m$ identity matrix.

- It is also the unique matrix $X \in \mathbb{R}^{n \times m}$ that satisfies the four *Moore–Penrose conditions*:
  1. $AXA = A$
  2. $XAX = X$
  3. $(AX)^{\top} = AX$
  4. $(XA)^{\top} = XA$

QR with Column Pivoting

- Another approach is to use QR with column pivoting, or QRCP.
- Suppose $A \in \mathbb{R}^{m \times n}$, and r be its rank. In **exact arithmetic**, QR with column pivoting is rank revealing if

$$Q^{\top} A \Pi = \begin{array}{cc} \begin{bmatrix} R_{11} & R_{12} \\ 0 & 0 \end{bmatrix} & \begin{array}{c} r \\ m-r \end{array} \\ \begin{array}{cc} r & n-r \end{array} & \end{array},$$

  where Π is a permutation matrix, and $\text{range}(A) = \text{span}\{q_1, \dots, q_r\}$.
- A rank-revealing QRCP is obtained by permuting columns such that the diagonal entry in R is maximized at each step.
- In particular, at the kth step,

$$(Q_{k-1} \cdots Q_1) A (\Pi_1 \cdots \Pi_{k-1}) = R^{(k-1)} = \begin{array}{cc} \begin{bmatrix} R_{11}^{(k-1)} & R_{12}^{(k-1)} \\ 0 & R_{22}^{(k-1)} \end{bmatrix} & \begin{array}{c} k-1 \\ m-k+1 \end{array} \\ \begin{array}{cc} k-1 & n-k+1 \end{array} & \end{array},$$

  where the column with the maximum 2-norm in $R_{22}^{(k-1)}$ is permuted to the kth column.

Solving Rank-Deficient Least Squares Problems by QRCP

- With rounding errors, one terminates if the computed $R_{22}^{(k-1)}$ ($\tilde{R}_{22}^{(k-1)}$) has a sufficiently small 2-norm compared to that of A.
  - If $\tilde{R}_{22}^{(k-1)}$ is small, then A is (numerically) rank deficient.
  - However, if $\mathrm{rank}(A) = k$, it does not follow that $\tilde{R}_{22}^{(k-1)}$ is small, so it may not reveal rank deficiency (and still lead to instability).
- For robustness, QRCP needs a condition number estimator to reveal the rank.

Rank-deficient least squares via truncated QRCP

Compute QRCP $AP = QR$ and estimate r.
Compute vector $c = Q_{:,1:r}^{\mathsf{T}} b$.
Solve triangular system $R_{1:r,1:r} y = c$ for y.
Set $x = P_{1:n,1:r} y$.

- Truncated QRCP is far less expensive than truncated SVD, and is robust with a good condition number estimator.
- Unlike SVD, QRCP uses a subset of columns of A.

Outline

1. Stability of Householder Triangularization (NLA §16,19)
2. Solution of Least Squares Problems
3. Rank-Deficient Least Squares Problems
4. **Software for Linear Algebra**

Software for Linear Algebra

- LAPACK: Linear Algebra PACKage (www.netlib.org/lapack/lug)
  - Standard library for solving linear systems and eigenvalue problems.
  - Successor of LINPACK (www.netlib.org/linpack) and EISPACK (www.netlib.org/eispack).
  - Depends on BLAS (Basic Linear Algebra Subprograms).
  - Parallel extensions include ScaLAPACK and PLAPACK (with MPI).
  - Note: Uses Fortran conventions for matrix arrangements.
- MATLAB
  - Factorization of A: `lu(A)` and `chol(A)`.
  - Solve $Ax = b$: $x = A\backslash b$.
    - Uses back/forward substitution for triangular matrices.
    - Uses Cholesky factorization for positive-definite matrices.
    - Uses LU factorization with partial pivoting for nonsymmetric matrices.
    - Uses Householder QR for least squares problems.
    - Uses some special routines for matrices with special sparsity patterns.
  - Uses LAPACK and other packages internally.
- Direct solvers for sparse matrices (e.g., SuperLU, SuiteSparse, MUMPS).

Some Commonly Used Functions

Example BLAS routines: Mat-vec multiplication: `dgemv`; Mat-mat multiplication: `dgemm`.

Table: Common Functions for Linear Systems

	LU Factorization		Solve linear system		Est. cond
	General	Symmetric	General	Symmetric	
LAPACK	`dgetrf`	`dpotrf/dsytrf`	**`dgesv`**	**`dposv/dposvx`**	`dgecon`
LINPACK	`dgefa`	`dpofa/dsifa`	`dgesl`	`dposl/dsisl`	`dgeco`
MATLAB	`lu`	`chol`	\	\	`rcond`

Table: Common Functions for Least Squares and Eigenvalues

	Linear least squares			Eigenvalue/vector		SVD
	QR	Solve	Rank-deficient	General	Symm.	
LAPACK	`dgeqrf`	**`dgels`**	**`dgelsy/dgelsd`**	**`dgeev`**	**`dsyev`**	**`dgesvd`**
LINPACK	`dqrdc`	`dqrsl`	`dqrst`	-	-	`dsvdc`
MATLAB	`qr`	\	\	`eig`	`eig`	`svd`

The first letter `s` stands for single-precision real, `d` for double-precision real, `c` for single-precision complex, and `z` for double-precision complex. Boldface routines are **driver** routines; others are **computational** routines.

Using LAPACK Routines in C Programs

- LAPACK was written in Fortran 77. Special attention is required when calling from C.
- Key differences between C and Fortran:
  1. Storage of matrices: column-major (Fortran) versus row-major (C/C++).
  2. Argument passing for subroutines in C and Fortran: pass by reference (Fortran) and pass by value (C/C++).
- Example C code (`example.c`) for solving a linear system using `sgesv`:
  - See the class website for sample code.
  - To compile, issue the command `cc -o example example.c -llapack -lblas`.
- Hint: To find a function name, refer to the LAPACK Users' Guide.
- To find out arguments for a given function, search on netlib.org.